

Sorting Algorithms

Class 24AI

The problem of sorting

Input: sequence $\langle a_1, a_2, \dots, a_n \rangle$ of numbers.

Output: permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

Example:

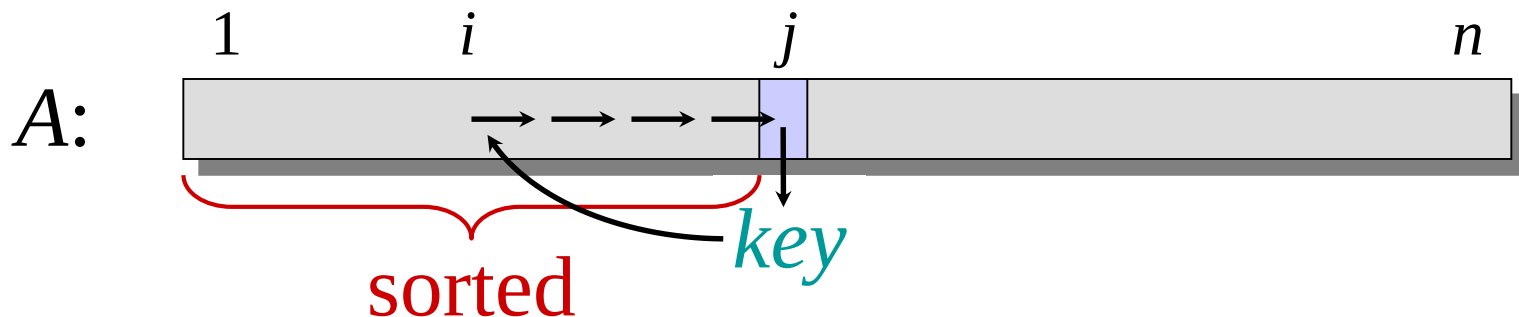
Input: 8 2 4 9 3 6

Output: 2 3 4 6 8 9

Insertion sort

“pseudocode”

```
INSERTION-SORT ( $A, n$ )    ▷  $A[1 \dots n]$   
  for  $j \leftarrow 2$  to  $n$   
    do  $key \leftarrow A[j]$   
       $i \leftarrow j - 1$   
      while  $i > 0$  and  $A[i] > key$   
        do  $A[i+1] \leftarrow A[i]$   
           $i \leftarrow i - 1$   
       $A[i+1] = key$ 
```



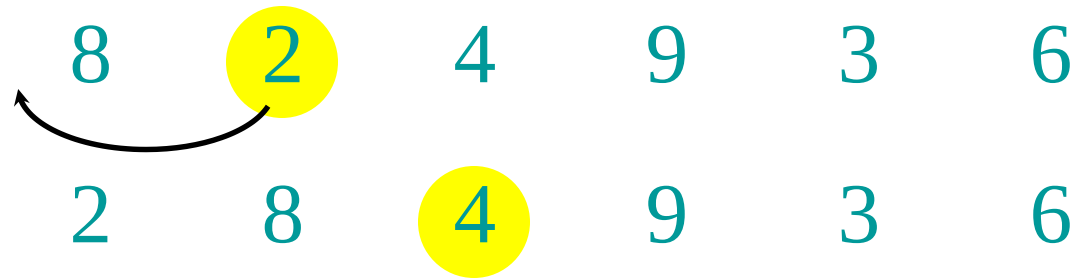
Example of insertion sort

8 2 4 9 3 6

Example of insertion sort



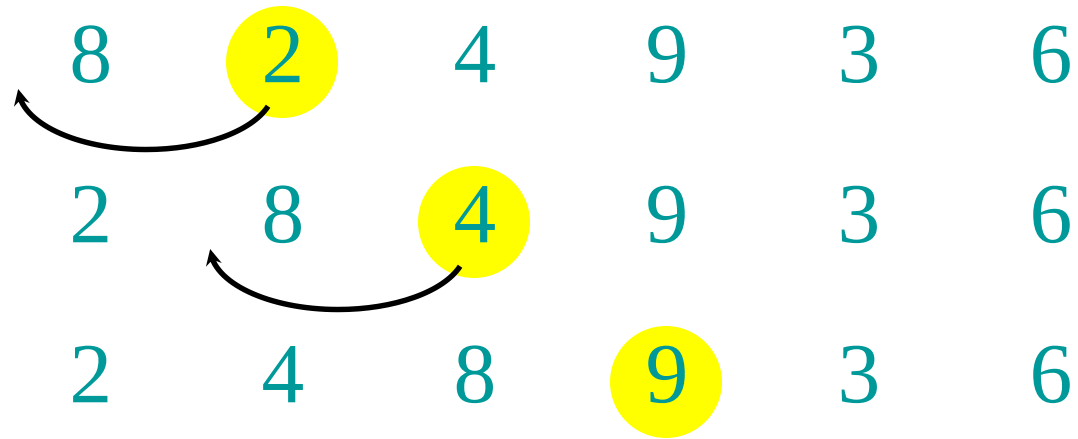
Example of insertion sort



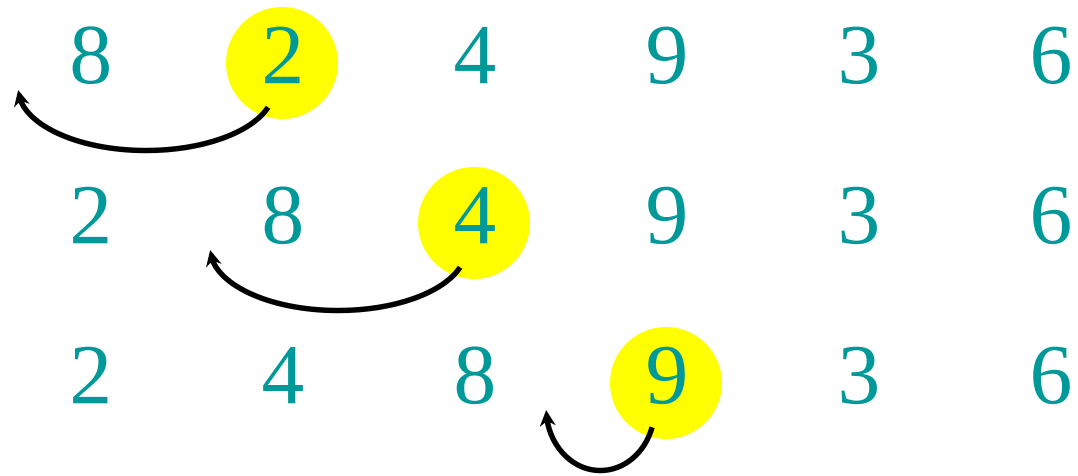
Example of insertion sort



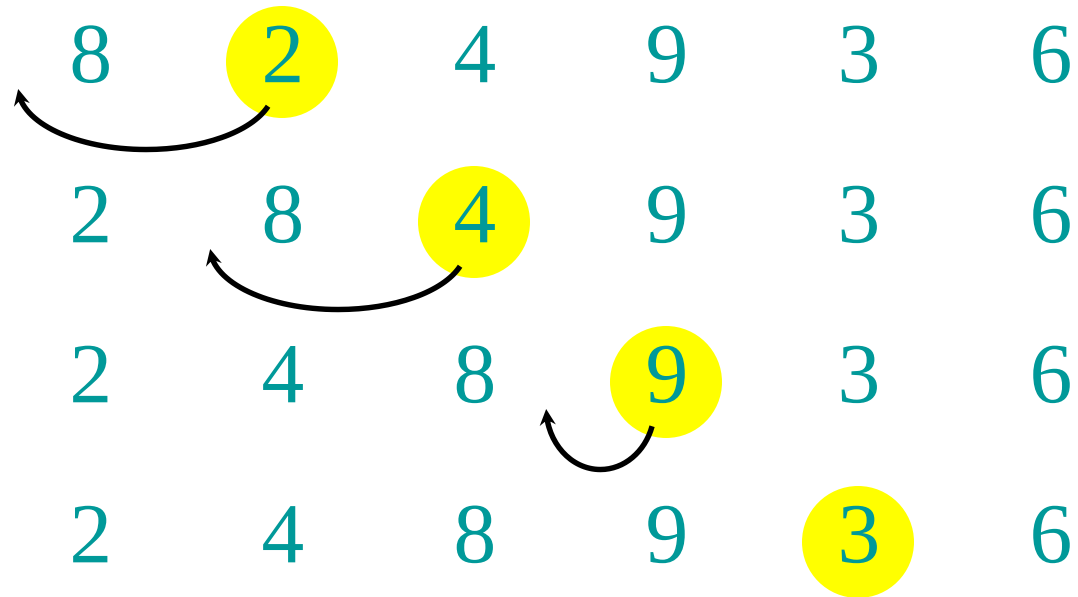
Example of insertion sort



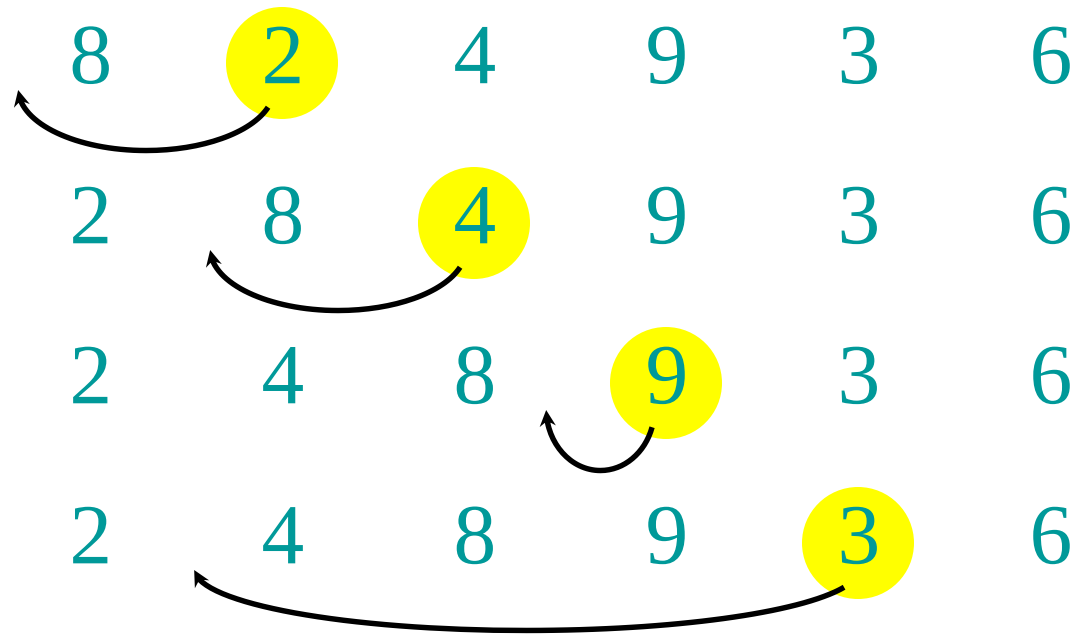
Example of insertion sort



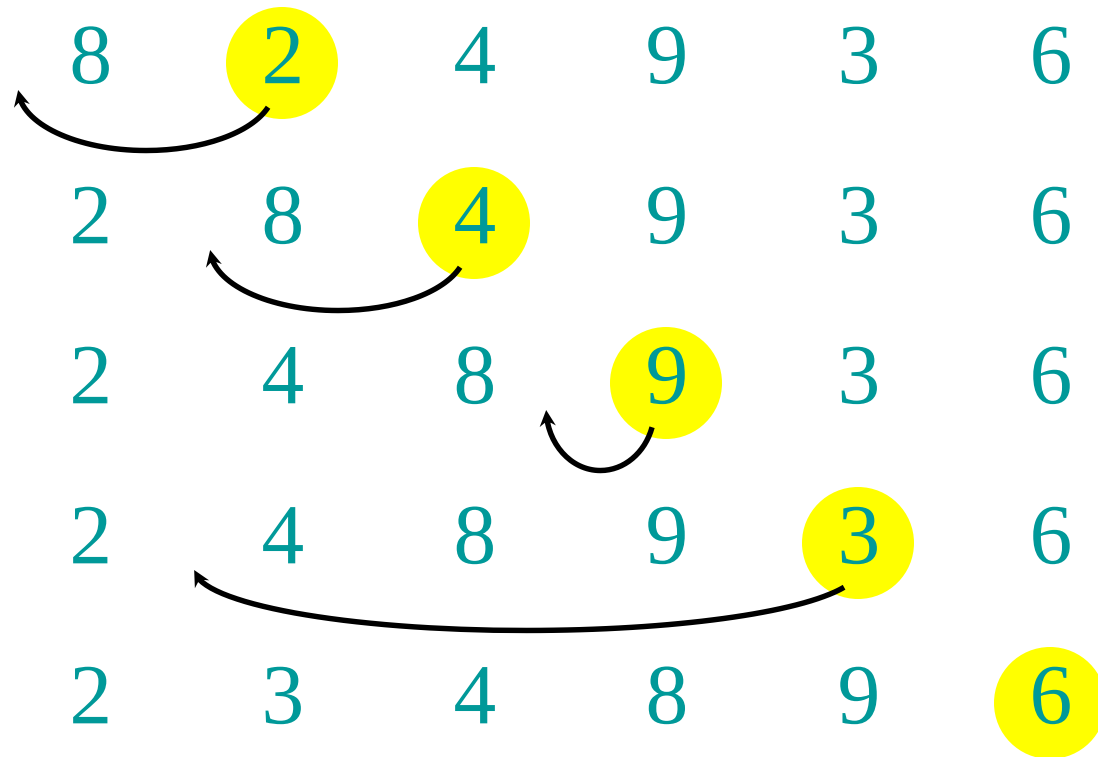
Example of insertion sort



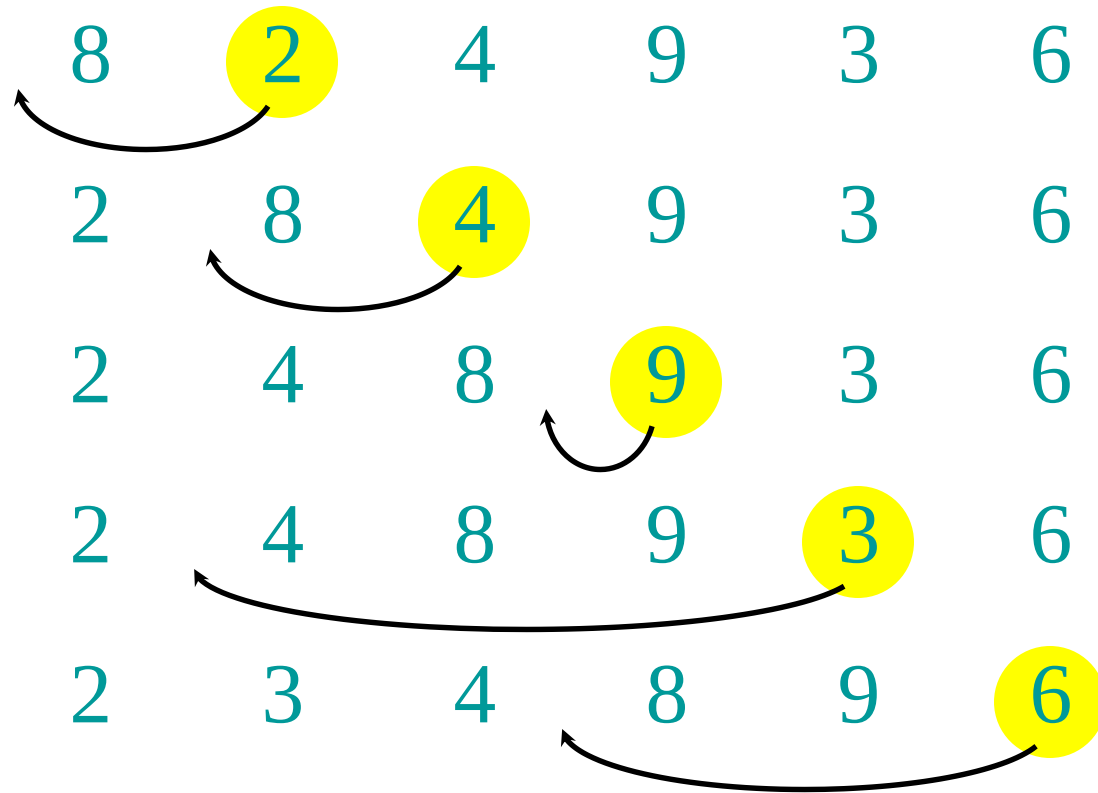
Example of insertion sort



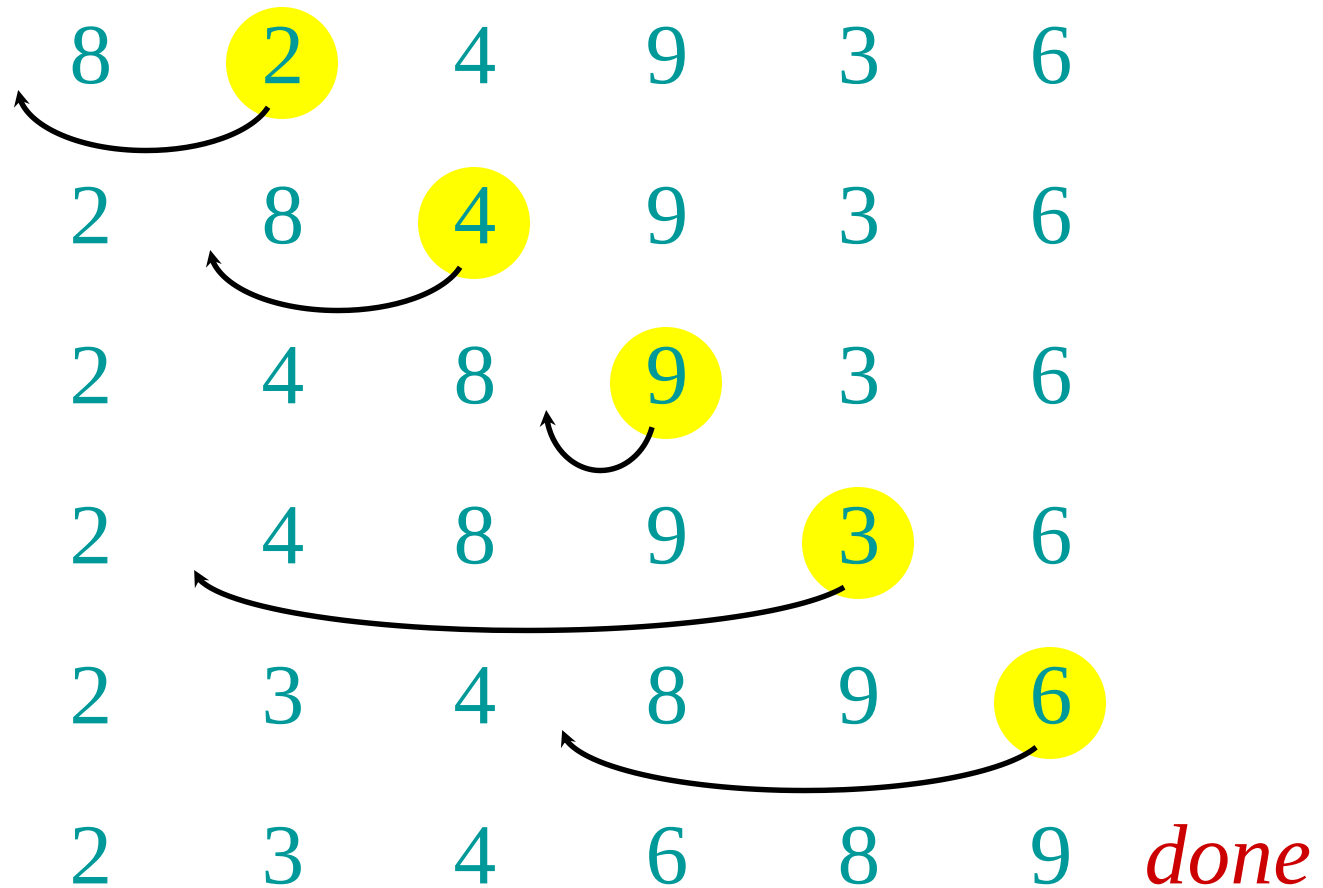
Example of insertion sort



Example of insertion sort



Example of insertion sort



Running time

- The running time depends on the input: an already sorted sequence is easier to sort.
- **Major Simplifying Convention:**
Parameterize the running time by the size of the input, since short sequences are easier to sort than long ones.
 - $T_A(n)$ = time of A on length n inputs
- Generally, we seek upper bounds on the running time, to have a guarantee of performance.

Kinds of analyses

Worst-case: (usually)

- $T(n)$ = maximum time of algorithm on any input of size n .

Average-case: (sometimes)

- $T(n)$ = expected time of algorithm over all inputs of size n .
- Need assumption of statistical distribution of inputs.

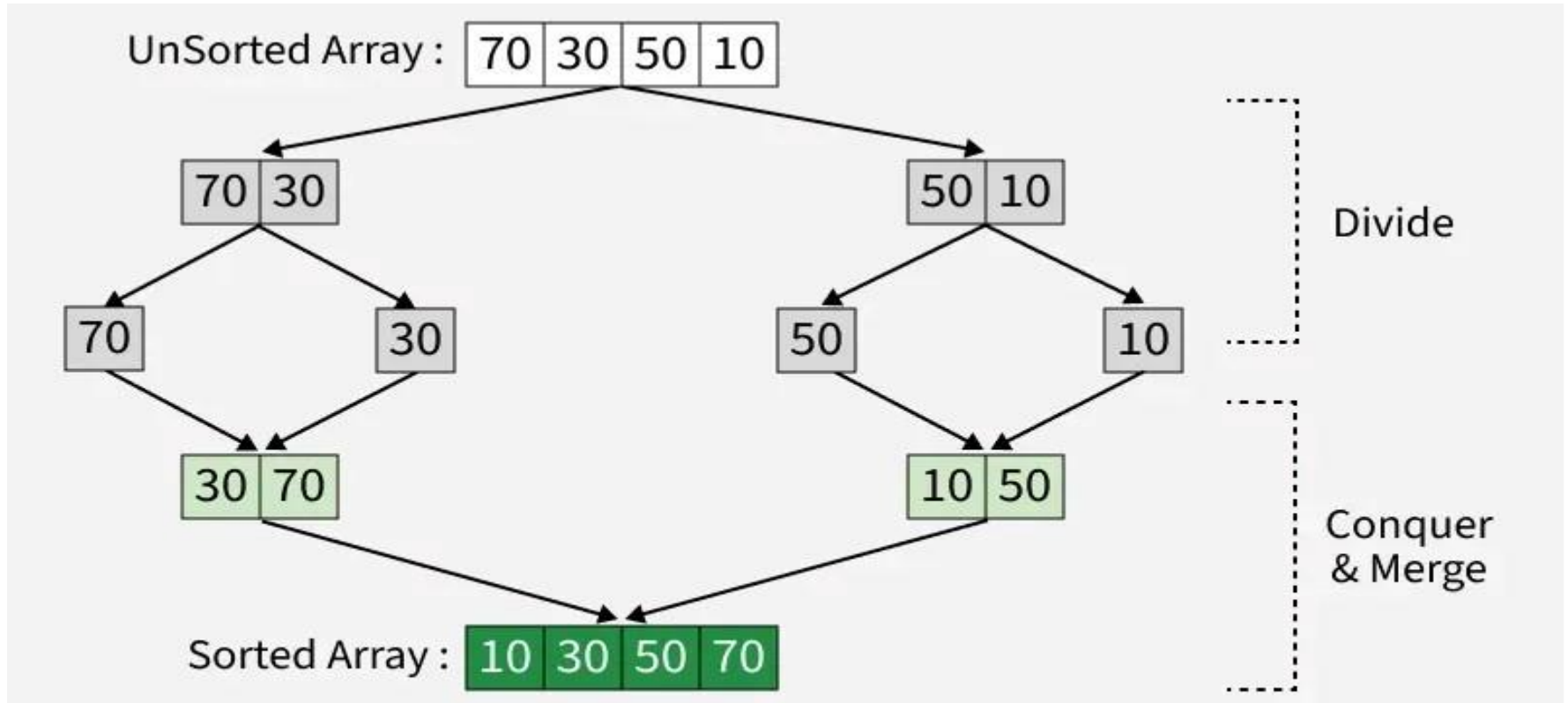
Best-case: (NEVER)

- Cheat with a slow algorithm that works fast on *some* input.

Merge sort

- Merge sort is a popular sorting algorithm known for its efficiency and stability.
- It follows the Divide and Conquer approach.
- It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Divide and Conquer Merge



Here's a step-by-step explanation of how merge sort works:

1.Divide: Divide the list or array recursively into two halves until it can no more be divided.

2.Conquer: Each subarray is sorted individually using the merge sort algorithm.

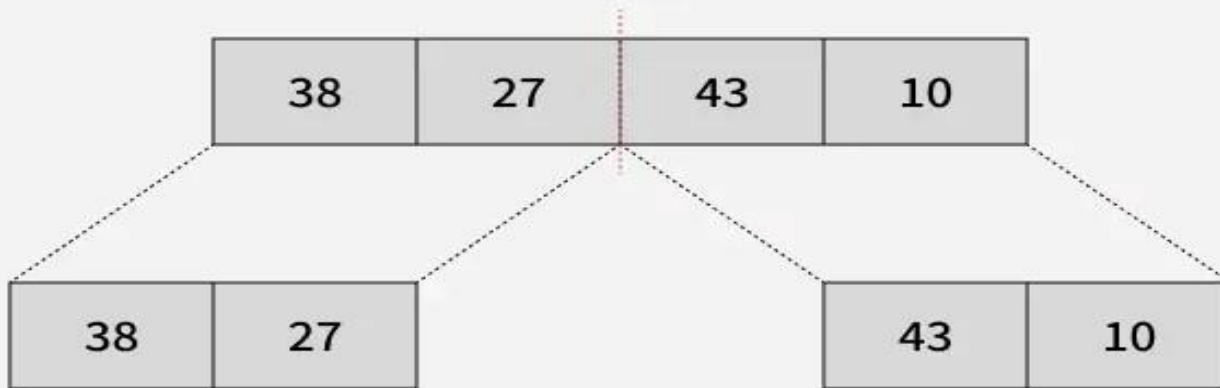
3.Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Let's sort the array or list [38, 27, 43, 10] using Merge Sort

01
Step

Splitting the Array into two equal halves

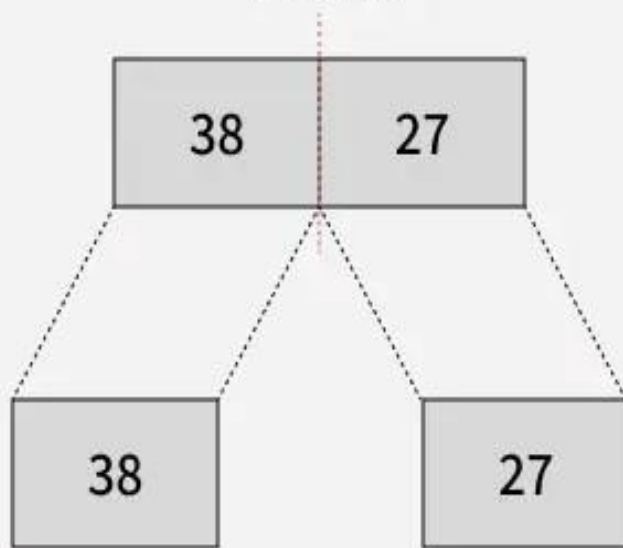
Divide



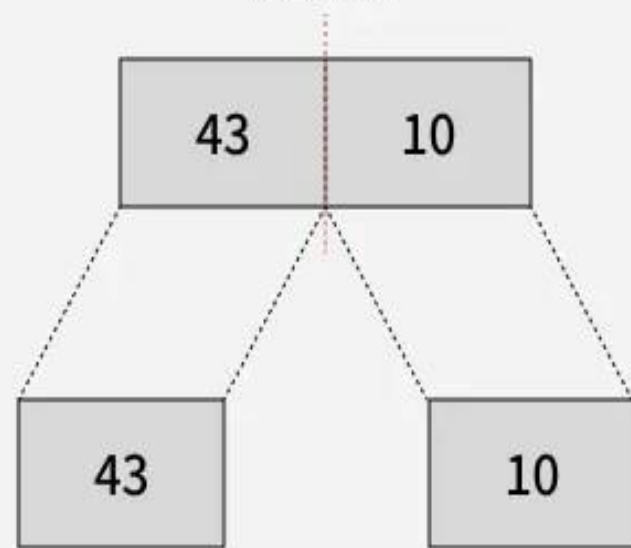
02
Step

Splitting the subarrays into two halves

Divide



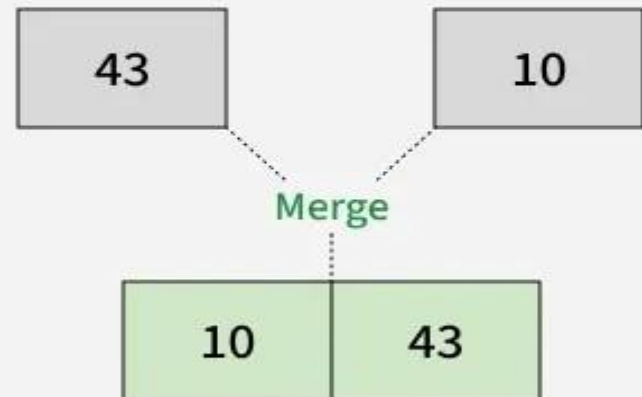
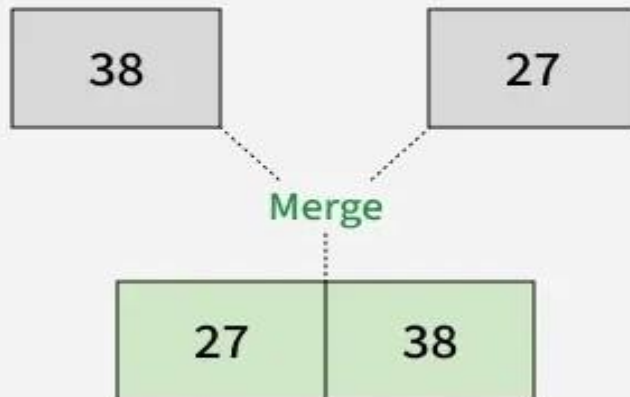
Divide



Sorted Subarrays

03
Step

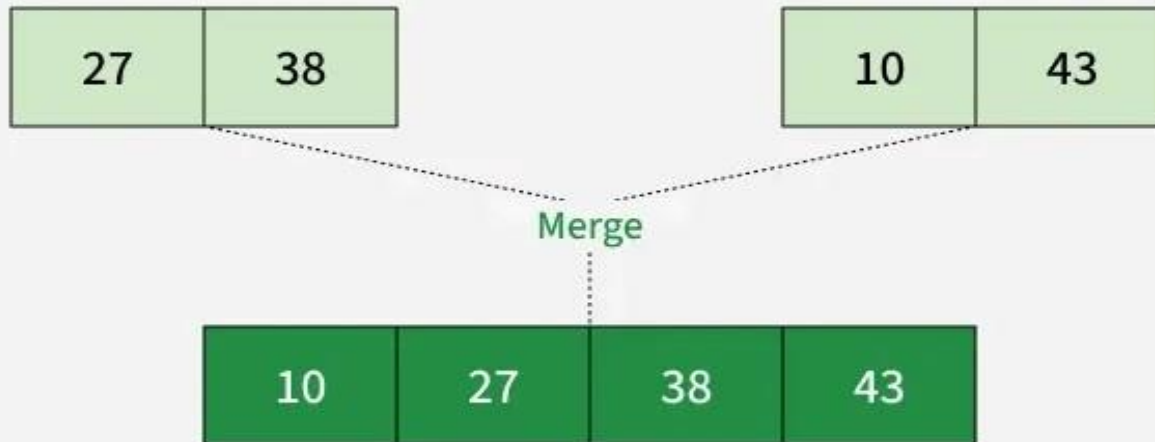
Merging unit length cells into sorted subarrays



Subarrays into sorted array

04
Step

Merging sorted subarrays into the sorted array



Let's look at the working of above example:

Divide:

- ***[38, 27, 43, 10]*** is divided into ***[38, 27]*** and ***[43, 10]*** .
- ***[38, 27]*** is divided into ***[38]*** and ***[27]*** .
- ***[43, 10]*** is divided into ***[43]*** and ***[10]*** .

Conquer:

- ***[38]*** is already sorted.
- ***[27]*** is already sorted.
- ***[43]*** is already sorted.
- ***[10]*** is already sorted.

Merge:

- Merge ***[38]*** and ***[27]*** to get ***[27, 38]*** .
- Merge ***[43]*** and ***[10]*** to get ***[10,43]*** .
- Merge ***[27, 38]*** and ***[10,43]*** to get the final sorted list ***[10, 27, 38, 43]***

Therefore, the sorted list is [10, 27, 38, 43] .

Complexity Analysis of Merge Sort

Time Complexity:

- **Best Case:** $O(n \log n)$, When the array is already sorted or nearly sorted.
- **Average Case:** $O(n \log n)$, When the array is randomly ordered.
- **Worst Case:** $O(n \log n)$, When the array is sorted in reverse order.

Auxiliary Space: $O(n)$, Additional space is required for the temporary array used during merging.

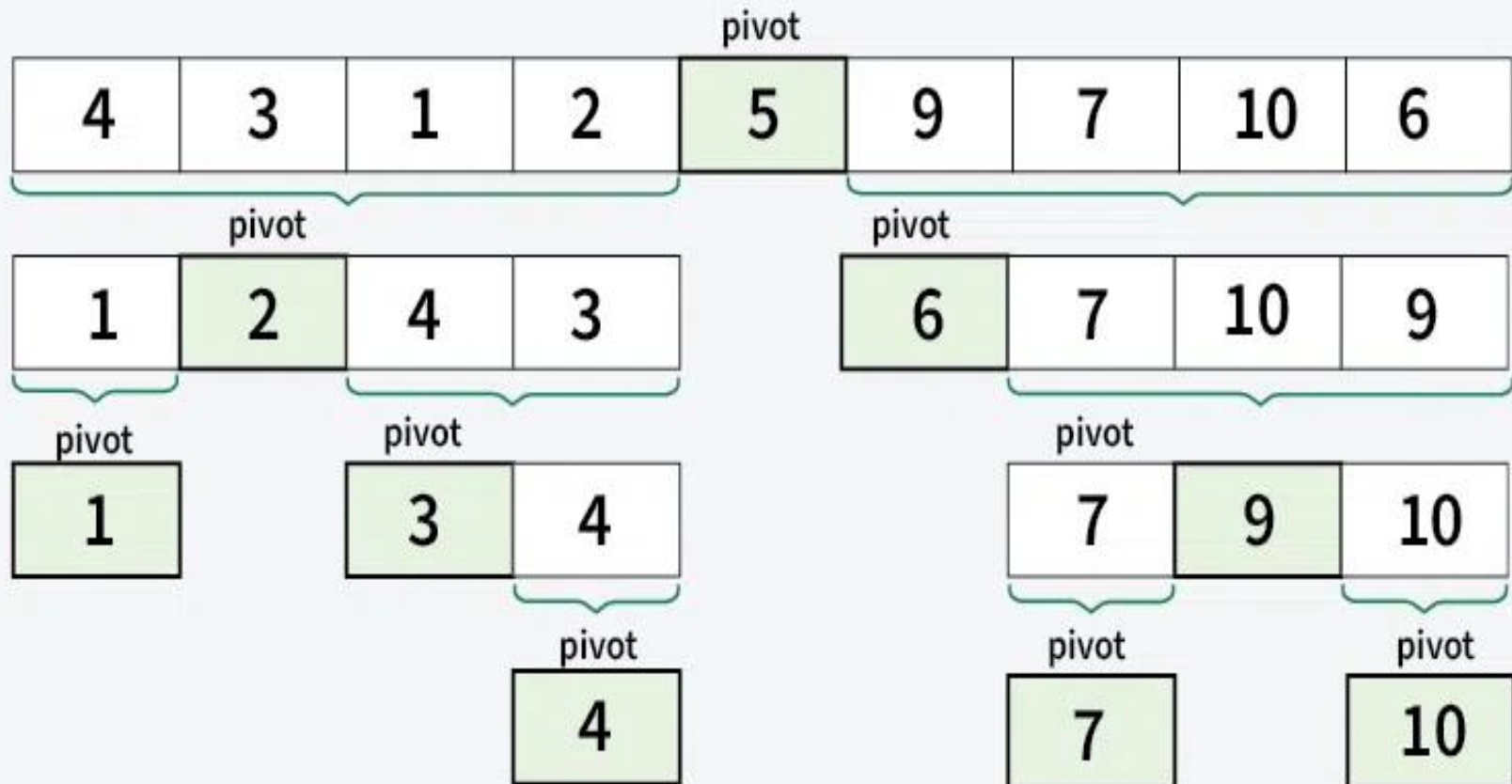
Quicksort

- Quicksort is a sorting algorithm based on the Divide and Conquer,
- Picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array. .

There are mainly three steps in the algorithm:

- 1.Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element,).
- 2.Partition the Array:** Re arrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right.
- 3.Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays.
- 4.Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Here, we have represented the recursive call after each partitioning step of the array.



Partition Algorithm

The key process in **quicksort** is a **partition()**.

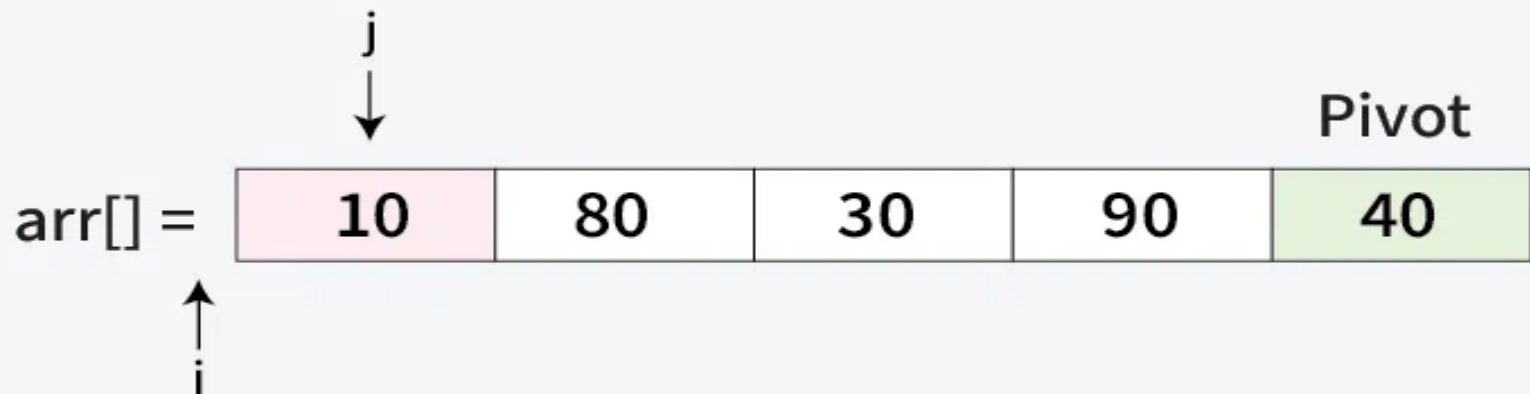
There are three common algorithms to partition. All these algorithms have $O(n)$ time complexity.

1. **Naive Partition**: Here we create copy of the array. First put all smaller elements and then all greater. Finally we copy the temporary array back to original array. This requires $O(n)$ extra space.
2. **Lomuto Partition**: We have used this partition in this article. This is a simple algorithm, we keep track of index of smaller elements and keep swapping. We have used it here in this article because of its simplicity.
3. **Hoare's Partition**: This is the fastest of all. Here we traverse array from both sides and keep swapping greater element on left with smaller on right while the array is not partitioned.

Let us understand the working of partition algorithm with the help of the following example:

01
Step

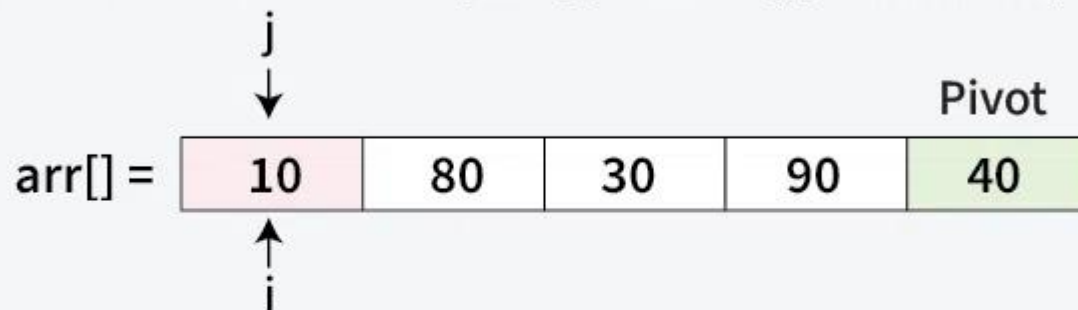
Pivot Selection: The last element $\text{arr}[4] = 40$ is chosen as the pivot.
Initial Pointers: $i = -1$ and $j = 0$.



Quick sort

02
Step

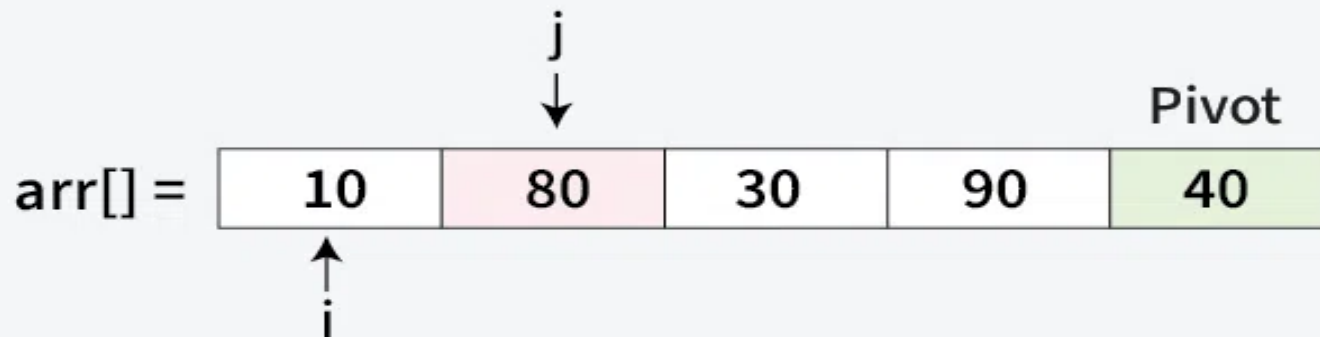
Since, $\text{arr}[j] < \text{pivot}$ ($10 < 40$)
Increment i to 0 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Quick sort

03
Step

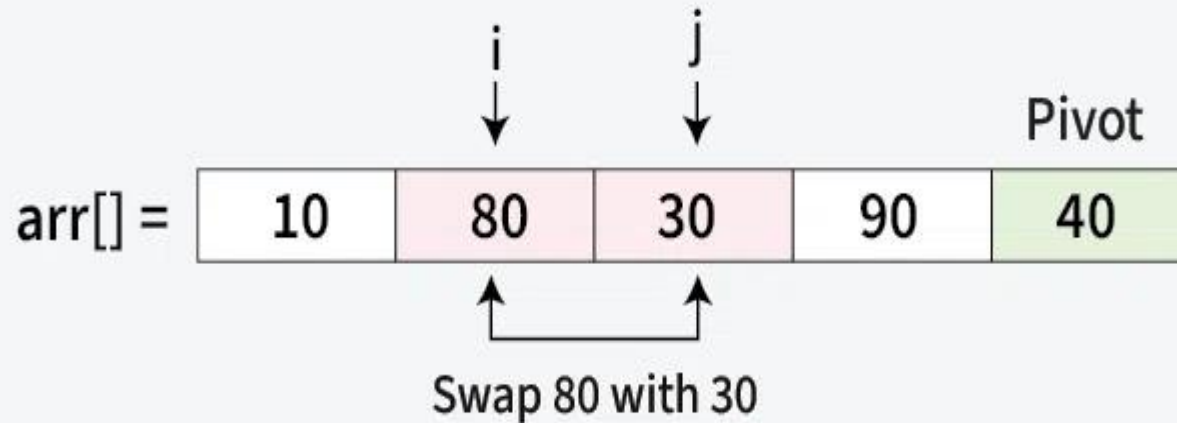
Since, $\text{arr}[j] > \text{pivot}$ ($80 > 40$)
No swap needed. Increment j by 1



Quick sort

04
Step

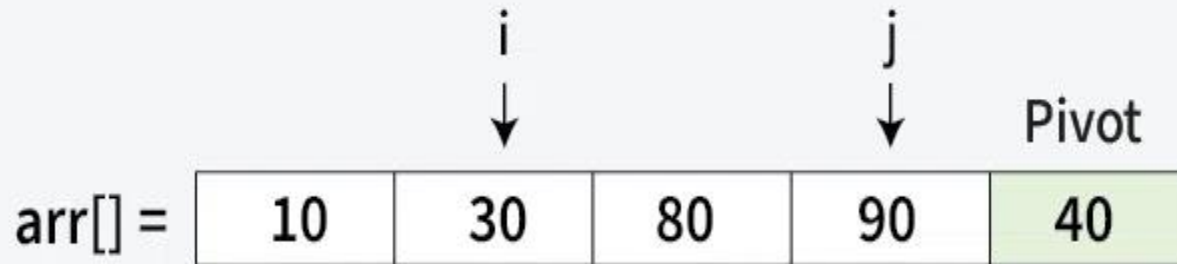
Since, $\text{arr}[j] < \text{pivot}$ ($30 < 40$)
Increment i by 1 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Quick sort

05
Step

Since, $\text{arr}[j] > \text{pivot}$ ($90 > 40$)
No swap needed. Increment j by 1

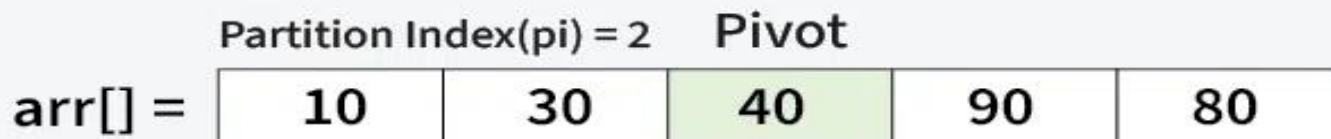
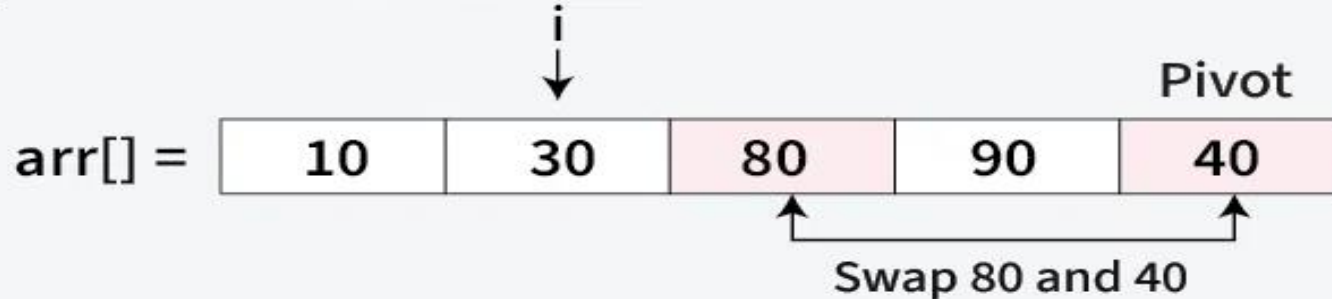


Quick sort

Move Pivot to its correct position

06
Step

Since traversal of j has ended. Now move pivot to its correct position, Swap $\text{arr}[i + 1] = \text{arr}[2]$ with $\text{arr}[4] = 40$.



Quick sort

Advantages of Quick Sort

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.
- It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.
- Fastest general purpose algorithm for large data when stability not required.

Disadvantages of Quick Sort

- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Applications of Quick Sort

- Sorting large datasets efficiently in memory.
- Used in library sort functions (like C++ and Java Arrays. Sort for primitives).
- Arranging records in databases for faster searching.
- Preprocessing step in algorithms requiring sorted input (e.g., binary search, two-pointer techniques).

Class Task

1. What is the running time of Merge-Sort if the array is already sorted? What is the best case running time of Merge-Sort?
2. Demonstrate the working of Partition on sequence (13, 5, 14, 11, 16, 12, 1, 15). What is the value of i returned at the completion of Partition?

- Bubble sort example on class board